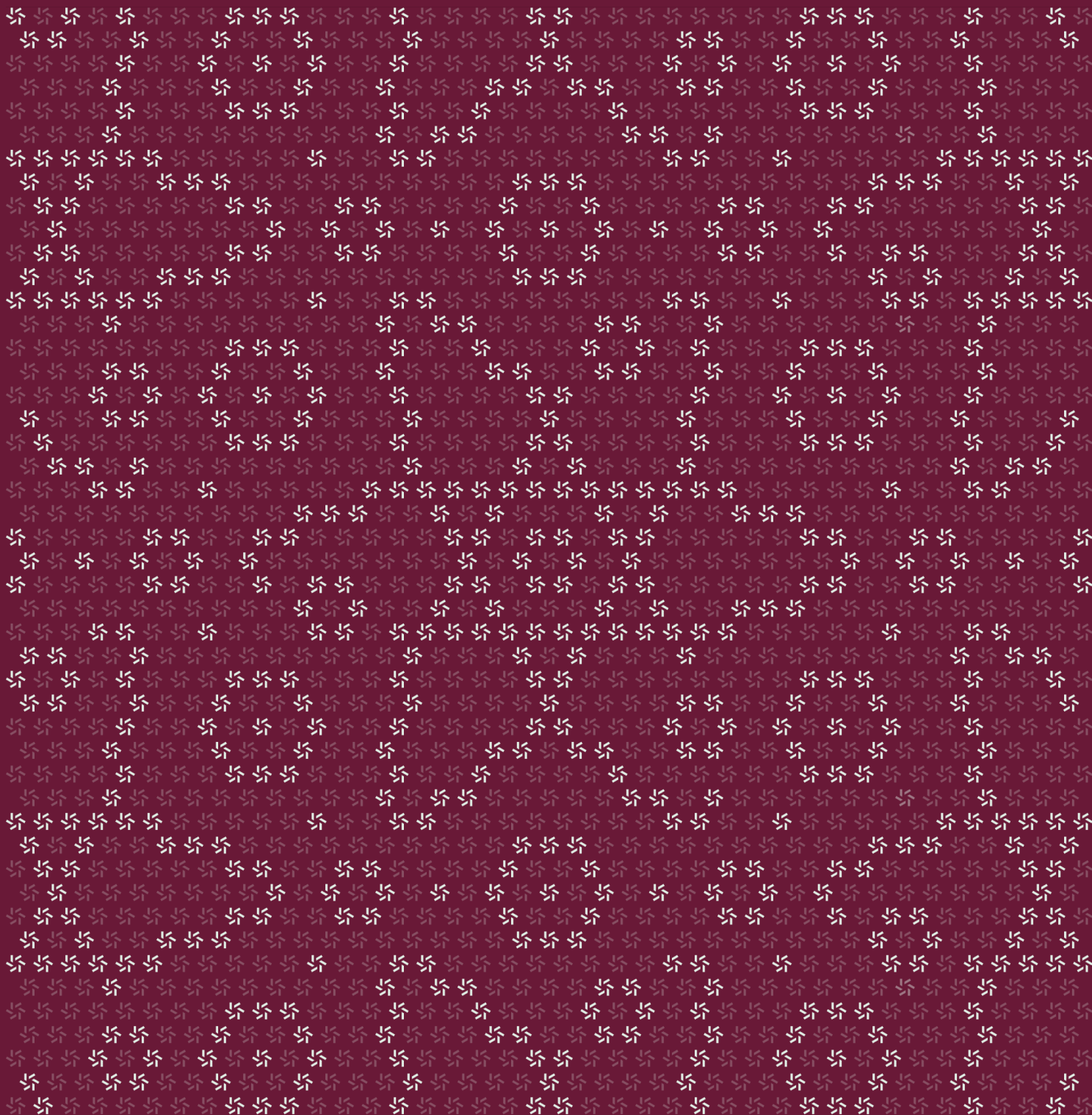


# IBC-Go GMP Module and Attestations Light Client

## Blockchain Security Assessment



## Contents

|                                                                                     |           |
|-------------------------------------------------------------------------------------|-----------|
| <b>About Zellic</b>                                                                 | <b>4</b>  |
| <hr/>                                                                               |           |
| <b>1. Overview</b>                                                                  | <b>4</b>  |
| 1.1. Executive Summary                                                              | 5         |
| 1.2. Goals of the Assessment                                                        | 5         |
| 1.3. Non-goals and Limitations                                                      | 5         |
| 1.4. Results                                                                        | 5         |
| <hr/>                                                                               |           |
| <b>2. Introduction</b>                                                              | <b>6</b>  |
| 2.1. About IBC-Go GMP Module and Attestations Light Client                          | 7         |
| 2.2. Methodology                                                                    | 7         |
| 2.3. Scope                                                                          | 9         |
| 2.4. Project Overview                                                               | 11        |
| 2.5. Project Timeline                                                               | 11        |
| <hr/>                                                                               |           |
| <b>3. Detailed Findings</b>                                                         | <b>11</b> |
| 3.1. Cross-chain signature replay via untagged attestation hash                     | 12        |
| 3.2. Attestation data lacks client-identifier binding, enabling cross-client replay | 14        |
| 3.3. The IsFrozen flag has no runtime activation path                               | 16        |
| 3.4. The InitGenesis function omits AccountsByAddress index population              | 18        |
| 3.5. The GMPPacketData.ValidateBasic function does not bound sender field length    | 20        |
| 3.6. Signature verification performs ECDSA recovery before quorum-size check        | 22        |
| 3.7. Zero-height guard present in Initialize and Validate is missing in UpdateState | 23        |

|                                    |                                                                                            |           |
|------------------------------------|--------------------------------------------------------------------------------------------|-----------|
| 3.8.                               | The sendPacket function is missing MsgResponses length check, inconsistent with executeMsg | 25        |
| 3.9.                               | Insufficient test coverage                                                                 | 27        |
| <hr data-bbox="490 493 1549 497"/> |                                                                                            |           |
| <b>4.</b>                          | <b>Threat Model</b>                                                                        | <b>28</b> |
| 4.1.                               | Module: GMP module                                                                         | 29        |
| 4.2.                               | Module: Attestations light client                                                          | 39        |
| <hr data-bbox="490 751 1549 756"/> |                                                                                            |           |
| <b>5.</b>                          | <b>Assessment Results</b>                                                                  | <b>52</b> |
| 5.1.                               | Disclaimer                                                                                 | 53        |

## About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website [zellic.io](https://zellic.io) and follow [@zellic\\_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at [hello@zellic.io](mailto:hello@zellic.io).



## 1. Overview

### 1.1. Executive Summary

Zellic conducted a security assessment for Cosmos Labs from February 19th, 2026 to February 24th, 2026. During this engagement, Zellic reviewed the IBC-Go GMP module and attestations light client's code for security vulnerabilities, design issues, and general weaknesses in security posture.

---

### 1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is GMP account generation vulnerable to denial of service or unauthorized account takeover?
  - Can a malicious GMP payload exploit the lack of encoding specification to cause unintended behavior?
  - Is the attestations light client vulnerable to replay or man-in-the-middle attacks?
- 

### 1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

---

### 1.4. Results

During our assessment on the scoped IBC-Go GMP Module and Attestations Light Client modules, we discovered nine findings. No critical issues were found. One finding was of high impact, two were of medium impact, two were of low impact, and the remaining findings were informational in nature.

Breakdown of Finding Impacts

| Impact Level  | Count |
|---------------|-------|
| Critical      | 0     |
| High          | 1     |
| Medium        | 2     |
| Low           | 2     |
| Informational | 4     |



## 2. Introduction

### 2.1. About IBC-Go GMP Module and Attestations Light Client

Cosmos Labs contributed the following description of the IBC-Go GMP module and attestations light client:

IBC-Go is an implementation of IBC Specs for Cosmos SDK chains. The GMP module is an IBC app that allows general message passing between chains. The attestation light client is a module which satisfies the light client interface, and uses a multisig instead of proofs to validate membership and non-membership.

---

### 2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

**Basic coding mistakes.** Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the modules.

**Nondeterminism.** Nondeterminism is a leading class of security issues on Cosmos. It can lead to consensus failure and blockchain halts. This includes but is not limited to vectors like wall-clock times, map iteration, and other sources of undefined behavior (UB) in Go.

**Arithmetic issues.** This includes but is not limited to integer overflows and underflows, floating-point associativity issues, loss of precision, and unfavorable integer rounding.

**Complex integration risks.** Several high-profile exploits have been the result of unintended consequences when interacting with the broader ecosystem, such as via IBC (Inter-Blockchain Communication Protocol). Zellic will review the project's potential external interactions and summarize the associated risks. If applicable, we will also examine any IBC interactions against the ICS Specification Standard to look for inconsistencies, flaws, and vulnerabilities.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

### 2.3. Scope

The engagement involved a review of the following targets:

#### IBC-Go GMP Module and Attestations Light Client Modules

|          |            |
|----------|------------|
| Type     | Go         |
| Platform | Cosmos SDK |

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Target     | ibc-go                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Repository | <a href="https://github.com/cosmos/ibc-go">https://github.com/cosmos/ibc-go</a>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Version    | 9ffc7482390229a84ca982d6af596689a1686756                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Programs   | apps/27-gmp/ibc_module.go<br>apps/27-gmp/module.go<br>apps/27-gmp/internal/events/events.go<br>apps/27-gmp/keeper/account.go<br>apps/27-gmp/keeper/genesis.go<br>apps/27-gmp/keeper/keeper.go<br>apps/27-gmp/keeper/msg_server.go<br>apps/27-gmp/keeper/query_server.go<br>apps/27-gmp/keeper/relay.go<br>apps/27-gmp/types/account.go<br>apps/27-gmp/types/ack.go<br>apps/27-gmp/types/autocli.go<br>apps/27-gmp/types/codec.go<br>apps/27-gmp/types/errors.go<br>apps/27-gmp/types/events.go<br>apps/27-gmp/types/expected_keepers.go<br>apps/27-gmp/types/genesis.go<br>apps/27-gmp/types/keys.go<br>apps/27-gmp/types/messages.go<br>apps/27-gmp/types/packet.go<br>apps/27-gmp/types/router.go<br>apps/27-gmp/types/solidity_abi.go<br>light-clients/attestations/abi.go<br>light-clients/attestations/attestation_proof.go<br>light-clients/attestations/client_state.go<br>light-clients/attestations/codec.go<br>light-clients/attestations/consensus_state.go<br>light-clients/attestations/doc.go<br>light-clients/attestations/errors.go<br>light-clients/attestations/keys.go<br>light-clients/attestations/light_client_module.go<br>light-clients/attestations/module.go<br>light-clients/attestations/signature.go<br>light-clients/attestations/store.go<br>light-clients/attestations/update.go |

## 2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.2 person-weeks. The assessment was conducted by two consultants over the course of four calendar days.

### Contact Information

---

The following project manager was associated with the engagement:

**Jacob Goreski**  
↗ Engagement Manager  
[jacob@zellic.io](mailto:jacob@zellic.io) ↗

---

The following consultants were engaged to conduct the assessment:

**Raj Agarwal**  
↗ Engineer  
[raj@zellic.io](mailto:raj@zellic.io) ↗

**Jisub Kim**  
↗ Engineer  
[jisub@zellic.io](mailto:jisub@zellic.io) ↗

---

## 2.5. Project Timeline

The key dates of the engagement are detailed below.

**February 19, 2026** Kick-off call

---

**February 19, 2026** Start of primary review period

---

**February 24, 2026** End of primary review period

### 3. Detailed Findings

#### 3.1. Cross-chain signature replay via untagged attestation hash

|            |                     |          |      |
|------------|---------------------|----------|------|
| Target     | Light-client module |          |      |
| Category   | Cryptography        | Severity | High |
| Likelihood | Medium              | Impact   | High |

#### Description

**Note:** This issue was reported by Cosmos Labs during the audit period. We have included it here for completeness and to document the fix.

The `verifySignatures()` function computes the hash over raw attestation bytes without a domain separator that binds the signature to a specific operation type:

```
// signature.go:L#31
hash := sha256.Sum256(proof.AttestationData)
```

This same function is invoked by three distinct operations: `VerifyClientMessage` (client state update), `verifyMembership`, and `verifyNonMembership`. Each operation expects a different ABI encoded payload type — `StateAttestation` for updates and `PacketAttestation` for packet proofs — but `verifySignatures` accepts any blob that satisfies the quorum threshold against the stored attester set.

An attacker can obtain a legitimately signed `PacketAttestation` blob from an on-chain membership proof transaction and submit it to `UpdateState` as a `ClientMessage`. Because `VerifyClientMessage` only calls `verifySignatures`, the quorum check passes. Then `UpdateState` calls `ABIDecodeStateAttestation` unconditionally:

```
// update.go:L#40-43
stateAttestation,
    err := ABIDecodeStateAttestation(attestationProof.AttestationData)
if err != nil {
    panic(fmt.Sprintf("failed to ABI decode attestation data: %v", err))
}
```

The ABI encoding of `PacketAttestation` uses a top-level dynamic type offset pointer as its blob; it interprets the offset pointer as `StateAttestation.Height` (yielding the fixed value 32) and interprets the packet-attested height as `StateAttestation.Timestamp`. The result is stored unconditionally:

```
// update.go:L#45-57
height := clienttypes.NewHeight(0, stateAttestation.Height) // always height
32
consensusState := &ConsensusState{
    Timestamp: stateAttestation.Timestamp,
}

setConsensusState(clientStore, cdc, consensusState, height)

if stateAttestation.Height > cs.LatestHeight {
    cs.LatestHeight = stateAttestation.Height
}

setClientState(clientStore, cdc, cs)
```

## Impact

Any signed `PacketAttestation` blob available on chain can be replayed by anyone to overwrite the consensus state at height 32 with a garbage timestamp. IBC operations that verify proofs at height 32 will receive an incorrect timestamp, causing packet time-outs to fire incorrectly or making those proofs unverifiable.

## Recommendations

Consider introducing a domain separator into the hash preimage that encodes both the attestation type and the client identifier before signature verification, so a `PacketAttestation` signature cannot satisfy verification for a `StateAttestation` context and vice versa.

## Remediation

This issue has been acknowledged by Cosmos Labs, and a fix was implemented in commit [73cea875](#).

### 3.2. Attestation data lacks client-identifier binding, enabling cross-client replay

|                   |                     |                 |        |
|-------------------|---------------------|-----------------|--------|
| <b>Target</b>     | Light-client module |                 |        |
| <b>Category</b>   | Cryptography        | <b>Severity</b> | Medium |
| <b>Likelihood</b> | Medium              | <b>Impact</b>   | Medium |

#### Description

Neither StateAttestation nor PacketAttestation include a client identifier in its ABI encoded payload:

```
// abi.go:L#47-57
type StateAttestation struct {
    Height    uint64
    Timestamp uint64
}

type PacketAttestation struct {
    Height    uint64
    Packets []PacketCompact
}
```

The hash over which attestors sign is derived solely from these bytes:

```
// signature.go:L#31
hash := sha256.Sum256(proof.AttestationData)
```

If two light-client instances are configured with overlapping or identical AttestorAddresses, a proof signed in the context of one client (client A) is structurally valid for any other client sharing those attestors (client B). There is no field in the signed data that binds an attestation to a specific client identifier, and verifySignatures performs no such check.

#### Impact

If the same attesor set serves multiple clients, a signed attestation for client A can be replayed against client B to update its state or forge membership proofs. The attacker only needs a previously submitted blob, which is observable on chain.

## Recommendations

Consider including the client identifier in the hash preimage used for signature verification, ensuring that signatures are bound to a specific client and cannot be transferred across client instances.

## Remediation

This issue has been acknowledged by Cosmos Labs.

### 3.3. The IsFrozen flag has no runtime activation path

|                   |                     |                 |        |
|-------------------|---------------------|-----------------|--------|
| <b>Target</b>     | Light-client module |                 |        |
| <b>Category</b>   | Completeness        | <b>Severity</b> | Medium |
| <b>Likelihood</b> | Low                 | <b>Impact</b>   | Medium |

#### Description

The `ClientState` struct exposes an `IsFrozen` field that is checked in `VerifyClientMessage`, `verifyMembership`, and `verifyNonMembership` to gate all client operations. However, no code path in the module sets `IsFrozen` to true at runtime.

The IBC misbehavior-detection interface is stubbed out entirely: `CheckForMisbehaviour` unconditionally returns `false`, and `UpdateStateOnMisbehaviour` panics if reached:

```
// light_client_module.go:L#75-82
func (LightClientModule) CheckForMisbehaviour(...) bool {
    return false
}

func (LightClientModule) UpdateStateOnMisbehaviour(...) {
    panic(errorsmod.Wrap(ibcerrors.ErrInvalidRequest,
        "updateStateOnMisbehaviour is not supported"))
}
```

The equivalent Solidity and Solana implementations of the attestations light client include a timestamp monotonicity check: if a newly submitted `StateAttestation` carries a timestamp inconsistent with previously stored consensus state, the client is frozen. This check is absent from the Go implementation. As a result, the `IsFrozen` field and the associated freeze-check branches in `VerifyClientMessage`, `verifyMembership`, and `verifyNonMembership` are unreachable dead code.

#### Impact

If attestors are compromised and submit conflicting state attestations, the client cannot be frozen automatically. IBC packet processing continues against a potentially invalid client state until governance acts.

## Recommendations

Consider implementing a timestamp monotonicity check in `VerifyClientMessage` that compares the incoming `StateAttestation` timestamp against the stored consensus state at the same or adjacent height, and sets `IsFrozen = true` on disagreement, consistent with the existing Solidity and Solana implementations.

## Remediation

This issue has been acknowledged by Cosmos Labs, and a fix was implemented in commit [34044e59](#) ↗.

### 3.4. The InitGenesis function omits AccountsByAddress index population

| Target     | GMP module      |          |     |
|------------|-----------------|----------|-----|
| Category   | Coding Mistakes | Severity | Low |
| Likelihood | High            | Impact   | Low |

#### Description

The `InitGenesis()` function restores ICS-27 accounts by writing only to the `Accounts` collection, keyed by `(clientId, sender, salt)`:

```
// keeper/genesis.go:L#25-30
if err := k.Accounts.Set(ctx, collections.Join3(account.AccountId.ClientId,
    account.AccountId.Sender, account.AccountId.Salt), types.ICS27Account{
    Address: account.AccountAddress,
    AccountId: &account.AccountId,
}); err != nil {
    return err
}
```

It does not populate the `AccountsByAddress` collection, which is the reverse index keyed by account address. The `GetAccount()` function reads exclusively from `AccountsByAddress`:

```
// keeper/account.go:L#76-83
func (k *Keeper) GetAccount(ctx context.Context, address sdk.AccAddress)
(*types.ICS27Account, error) {
    ics27Acc, err := k.AccountsByAddress.Get(ctx, address)
    if err != nil {
        return nil, err
    }

    return &ics27Acc, nil
}
```

#### Impact

After a genesis import or chain restart, the `AccountIdentifier` query fails for all imported accounts. Relayers and tooling that rely on address-to-identifier lookups will return errors until each account is first touched by a packet receive.

## Recommendations

Consider adding a corresponding `k.AccountsByAddress.Set` call inside the `InitGenesis` loop, mirroring the pattern used in `getOrCreateICS27Account`.

## Remediation

This issue has been acknowledged by Cosmos Labs, and a fix was implemented in commit [5a50845b](#).

### 3.5. The `GMPPacketData.ValidateBasic` function does not bound sender field length

|                   |               |                 |               |
|-------------------|---------------|-----------------|---------------|
| <b>Target</b>     | GMP module    |                 |               |
| <b>Category</b>   | Code Maturity | <b>Severity</b> | Informational |
| <b>Likelihood</b> | N/A           | <b>Impact</b>   | Informational |

#### Description

The `GMPPacketData.ValidateBasic()` function checks that the `Sender` field is nonempty but applies no upper length bound, unlike the `Receiver`, `Payload`, `Salt`, and `Memo` fields, which are each checked against named constants:

```
// types/packet.go:L#42-60
func (p GMPPacketData) ValidateBasic() error {
    if strings.TrimSpace(p.Sender) == "" {
        return errorsmod.Wrap(ibcerrors.ErrInvalidAddress, "missing sender address")
    }
    // receiver is allowed to be empty
    if len(p.Receiver) > MaximumReceiverLength { ... }
    if len(p.Payload) > MaximumPayloadLength { ... }
    if len(p.Salt) > MaximumSaltLength { ... }
    if len(p.Memo) > MaximumMemoLength { ... }
    ...
}
```

Outgoing packets pass through `MsgSendCall.ValidateBasic()`, which enforces Bech32 address validation on `Sender` and implicitly bounds its length. For packets received from remote chains, the `Sender` field is not subject to Bech32 validation, and `GMPPacketData.ValidateBasic()` provides the only check.

#### Impact

No practical exploit exists under routing as of the time of writing, as the `Sender` length is implicitly constrained by the protobuf encoding limit of the outer packet and by Bech32 validation on the outgoing path. The inconsistency is a defense-in-depth gap that could become relevant if `GMPPacketData.ValidateBasic()` is used independently in future code paths.

## Recommendations

Consider adding a `MaximumSenderLength` constant and a corresponding length check for `Sender` in `GMPPacketData.ValidateBasic()` for consistency with the other bounded fields.

## Remediation

This issue has been acknowledged by Cosmos Labs, and a fix was implemented in commit [cda845a3](#).

### 3.6. Signature verification performs ECDSA recovery before quorum-size check

|                   |                     |                 |               |
|-------------------|---------------------|-----------------|---------------|
| <b>Target</b>     | Light-client module |                 |               |
| <b>Category</b>   | Optimization        | <b>Severity</b> | Informational |
| <b>Likelihood</b> | N/A                 | <b>Impact</b>   | Informational |

#### Description

The `verifySignatures()` function iterates over all provided signatures and performs ECDSA public-key recovery for each before checking whether the total number of signatures meets the quorum threshold:

```
// signature.go:L#34-64
for i, sig := range proof.Signatures {
    // ECDSA recovery per signature ...
}
if len(proof.Signatures) < int(cs.MinRequiredSigs) {
    return errorsmod.Wrapf(ErrInvalidQuorum, "quorum not met: required %d, got %d", ...)
}
```

#### Impact

Any caller can submit a proof with fewer signatures than `MinRequiredSigs`, causing the node to perform up to `len(proof.Signatures)` ECDSA recoveries before returning an error. The quorum check at the end of the loop still prevents acceptance of under-quorum proofs, so there is no security impact. The concern is limited to unnecessary computation.

#### Recommendations

Consider moving the `len(proof.Signatures) < int(cs.MinRequiredSigs)` check to the top of `verifySignatures`, before the signature iteration loop.

#### Remediation

This issue has been acknowledged by Cosmos Labs, and a fix was implemented in commit [5c8cf871](#).

### 3.7. Zero-height guard present in Initialize and Validate is missing in UpdateState

|                   |                     |                 |               |
|-------------------|---------------------|-----------------|---------------|
| <b>Target</b>     | Light-client module |                 |               |
| <b>Category</b>   | Code Maturity       | <b>Severity</b> | Informational |
| <b>Likelihood</b> | N/A                 | <b>Impact</b>   | Informational |

#### Description

Both `Initialize()` and `ClientState.Validate()` reject a height of zero:

```
// light_client_module.go:L#52-54
if clientState.LatestHeight == 0 {
    return errorsmod.Wrap(clienttypes.ErrInvalidClient, "initial height must be non-zero")
}
```

```
// client_state.go:L#67-69
if cs.LatestHeight == 0 {
    return errorsmod.Wrap(clienttypes.ErrInvalidClient, "latest height must be greater than 0")
}
```

However, `UpdateState()` omits this check when decoding the attested height:

```
// update.go:L#45-54
height := clienttypes.NewHeight(0, stateAttestation.Height)
consensusState := &ConsensusState{Timestamp: stateAttestation.Timestamp}
setConsensusState(clientStore, cdc, consensusState, height)
if stateAttestation.Height > cs.LatestHeight {
    cs.LatestHeight = stateAttestation.Height
}
```

#### Impact

No practical exploit exists, as reaching `UpdateState` requires a quorum-signed attestation and honest attestors would not sign a zero-height state. The missing check is a code consistency gap relative to `Initialize` and `Validate`.

## Recommendations

Consider adding a zero-height check at the top of `UpdateState`, consistent with the guards in `Initialize` and `Validate`.

## Remediation

This issue has been acknowledged by Cosmos Labs, and fixes were implemented in the following commits:

- [f0732e3f](#) ↗
- [ac0e2691](#) ↗

### 3.8. The sendPacket function is missing MsgResponses length check, inconsistent with executeMsg

|                   |               |                 |               |
|-------------------|---------------|-----------------|---------------|
| <b>Target</b>     | GMP module    |                 |               |
| <b>Category</b>   | Code Maturity | <b>Severity</b> | Informational |
| <b>Likelihood</b> | N/A           | <b>Impact</b>   | Informational |

#### Description

The executeMsg() function in relay.go checks the length of res.MsgResponses before indexing:

```
// keeper/relay.go:L#145-148
if len(res.MsgResponses) != 1 {
    return nil, errorsmod.Wrapf(ibcerrors.ErrLogic, "expected exactly one Msg response for msg %s, got %d", sdk.MsgTypeURL(msg), len(res.MsgResponses))
}
msgResponse := res.MsgResponses[0]
```

The sendPacket() function in msg\_server.go indexes res.MsgResponses directly without a prior length check:

```
// keeper/msg_server.go:L#72-75
msgResponse := res.MsgResponses[0]
if msgResponse == nil {
    return 0, errorsmod.Wrapf(ibcerrors.ErrLogic, "got nil Msg response for msg %s", sdk.MsgTypeURL(msg))
}
```

#### Impact

No practical exploit exists, as the IBC core MsgSendPacket handler guarantees exactly one response. The omission is a code consistency gap relative to executeMsg.

#### Recommendations

Consider adding a len(res.MsgResponses) != 1 check in sendPacket, consistent with the pattern in executeMsg.

## Remediation

This issue has been acknowledged by Cosmos Labs, and a fix was implemented in commit [ca89b231](#) ↗.

### 3.9. Insufficient test coverage

|                   |               |                 |     |
|-------------------|---------------|-----------------|-----|
| <b>Target</b>     | Project-wide  |                 |     |
| <b>Category</b>   | Code Maturity | <b>Severity</b> | Low |
| <b>Likelihood</b> | N/A           | <b>Impact</b>   | Low |

#### Description

When building a complex contract with multiple moving parts (state changes) and dependencies, comprehensive testing is essential. This includes testing for both positive and negative scenarios on every function that touches the contract's state.

Positive tests should verify that each function's side effect is as expected, while negative tests should cover every revert, preferably in every logical branch.

The test coverage for this project should be expanded to include all contracts, and all functions — not just surface-level functions. It is important to test the invariants required for ensuring security and also verify any mathematical properties from the project's specification. Additionally, testing cross-chain function calls and transfers is recommended to ensure the desired functionality.

While the project includes test files for all in-scope modules, the existing tests largely validate the happy path and do not cover adversarial scenarios. Missing cases include submitting a `PacketAttestation` blob through `UpdateState`, replaying an attestation across clients sharing the same attester set, passing a zero-height `StateAttestation` to `UpdateState`, and querying `AccountIdentifier` after a genesis import. Several of the findings in this report could have been surfaced earlier had these scenarios been covered.

#### Impact

Good test coverage has multiple effects:

- It finds bugs and design flaws early (preaudit or prerelease).
- It gives insight into areas for optimization (e.g., gas cost).
- It displays code maturity.
- It bolsters customer trust in the product.
- It improves understanding of how the code functions, integrates, and operates — for developers and auditors alike.
- It increases development velocity long-term.

The last point may seem contradictory given the time investment to create and maintain tests. However, tests help developers trust their own changes. It is difficult to know if a code refactor — or even just a small one-line fix — breaks other code if there are no tests. This is especially true for

new developers or those returning to the code after a prolonged absence.

Tests have your back here. They are an excellent indicator that the existing functionality was most likely not broken by a change to the code. Without a comprehensive test suite, the above benefits are lost. This increases the likelihood of bugs and vulnerabilities going unnoticed until after deployment, which can be costly and damaging to the project's reputation.

## **Recommendations**

We recommend expanding the test suite to ensure extensive and thorough test coverage.

## **Remediation**

This issue has been acknowledged by Cosmos Labs.

## 4. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the modules and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

### 4.1. Module: GMP module

This section provides a comprehensive threat model for the GMP module, detailing the intended behavior, preconditions, inputs, and potential risks for the module's core functions.

#### GMP overview

The 27-gmp module implements ICS27-GMP, a cross-chain general message passing protocol built on IBC V2. It allows any account on a source chain to invoke smart contract calls on a destination chain by submitting a `MsgSendCall`, which is serialized as a `GMPPacketData` and dispatched as an IBC V2 packet over a single shared `gmpport`. Unlike ICS-27 Interchain accounts, there is no channel handshake or account preregistration. The destination chain deterministically derives a caller account from `(dest_client_id, sender, salt)` on first use and executes the opaque payload against it (handled by the destination chain), returning results via a mandatory acknowledgment.

#### MsgServer

**Function:** `SendCall(goCtx context.Context, msg *types.MsgSendCall)`

*Module:* `modules/apps/27-gmp/keeper/msg_server.go`

#### Intended behavior

The `SendCall` function is the public message-server handler for `MsgSendCall` through which any account on the source chain initiates a cross-chain contract call. The function validates the caller's address, constructs and validates the `GMPPacketData` from the message fields, delegates to the internal `sendPacket` helper to serialize and dispatch an IBC V2 packet, and returns the assigned packet sequence number to the caller.

#### Preconditions

- `msg.Sender` must be a valid Bech32-encoded Cosmos SDK account address.
- `msg.SourceClient` must be a valid IBC client identifier (validated by `host.ClientIdentifierValidator` inside `GMPPacketData.ValidateBasic` via the `MsgSendCall` message, and later inside `MsgSendPacket.ValidateBasic` inside the IBC

core handler).

- `msg.Receiver` must not exceed 2,048 bytes.
- `msg.Payload` must not exceed 32,768 bytes.
- `msg.Salt` must not exceed 32 bytes.
- `msg.Memo` must not exceed 32,768 bytes.
- `msg.TimeoutTimestamp` must be nonzero.
- `msg.Encoding` must be one of the following: "application/x-protobuf", "application/json", or "application/x-solidity-abi".
- A handler for `MsgSendPacket` (IBC V2 core) must be registered in the module's `msgRouter`.

### Inputs

- `SourceClient` string — a local IBC client identifier used to route the outgoing packet
- `Sender` string — the Bech32 address of the message signer on the source chain, used as the GMP account derivation input on the destination
- `Receiver` string — a destination-specific receiver identifier, which may be an EVM address, Solana program ID, or left empty
- `Salt []byte` — arbitrary bytes that differentiate GMP accounts for the same (`client_id`, `sender`) pair, used in deterministic account derivation on the destination
- `Payload []byte` — opaque calldata interpreted by the destination runtime (e.g., ABI-encoded EVM calldata, protobuf-encoded SDK messages)
- `TimeoutTimestamp uint64` — the absolute packet expiry time in nanoseconds since Unix epoch
- `Memo` string — optional metadata, which may carry callback instructions
- `Encoding` string — the serialization format for `GMPPacketData` — empty string defaults to "application/x-solidity-abi" in `sendPacket`

### Branching behavior

- The sender address is validated first; the function returns immediately if it is not a valid Bech32 address.
- The `GMPPacketData` is constructed from the message fields and independently validated against size and content constraints.
- If validation fails, the error is wrapped with the module version string and returned.
- The packet is dispatched via the internal `sendPacket` helper; any error is returned directly to the caller.
- On success, an info log is emitted, and the assigned packet sequence number is returned.

### External call analysis

- `packetData.ValidateBasic()` — field validation on the `GMPPacketData` struct
- `k.sendPacket(...)` — an internal helper that serializes the packet data, constructs an IBC V2 `MsgSendPacket`, and dispatches it through the message router, ultimately writing a packet commitment to the IBC core store and emitting IBC send events
- `k.Logger(ctx).Info(...)` — writes a structured log entry — no state change

**Function:** `sendPacket(ctx sdk.Context, encoding, sourceClient string, timeoutTimestamp uint64, packetData types.GMPPacketData)`

*Module:* `modules/apps/27-gmp/keeper/msg_server.go`

### Intended behavior

The `sendPacket` function is an internal helper that performs the low-level IBC V2 packet dispatch. It defaults the encoding to ABI if none was specified, marshals the `GMPPacketData` into bytes using the chosen encoding, assembles an IBC V2 `Payload` (with fixed port IDs of "gmpport" on both sides and version "ics27-2"), wraps it in a `MsgSendPacket`, and dispatches it through the message router. After the dispatch, it propagates the events emitted by the nested handler back to the calling context's `EventManager` and extracts and returns the assigned packet sequence number.

### Preconditions

- The encoding is either empty (will be normalized to "application/x-solidity-abi") or one of the three supported encoding strings.
- The `sourceClient` must be a valid IBC client identifier (validated again inside `MsgSendPacket.ValidateBasic`, executed by the IBC core handler).
- The `timeoutTimestamp` must be nonzero.
- A handler must be registered in `k.msgRouter` for `*channeltypesv2.MsgSendPacket`; otherwise, the function returns `ErrInvalidRequest`.
- The IBC core `SendPacket` handler must succeed (client active, no duplicate sequence, etc.).
- Also, `MarshalPacketData` requires the version string to equal "ics27-2".

### Inputs

- `encoding string` — the serialization format for `GMPPacketData` — an empty string is normalized to "application/x-solidity-abi"
- `sourceClient string` — the local IBC V2 client identifier identifying the counterparty chain

- `timeoutTimestamp uint64` — absolute packet time-out in nanoseconds since Unix epoch
- `packetData types.GMPPacketData` — the fully constructed and prevalidated GMP packet data struct containing Sender, Receiver, Salt, Payload, and Memo

### Branching behavior

- If no encoding is provided, it is silently defaulted to ABI encoding before marshalling.
- The packet data is serialized; any marshalling failure is returned immediately.
- If no handler is registered in the message router for the IBC V2 send-packet message, the function returns an invalid-request error without attempting dispatch.
- The IBC core handler is invoked; any failure (e.g., expired client, store error) is returned immediately.
- After a successful dispatch, events produced by the handler are reemitted into the outer context to prevent them from being silently dropped.
- The handler response is expected to contain exactly one message reply; if the reply is nil, a logic error is returned. The reply is then deserialized to extract the packet sequence number, which is returned to the caller.

### External call analysis

- `types.MarshalPacketData(&packetData, types.Version, encoding)` — stateless serialization of `GMPPacketData`, which dispatches to one of three codecs — for `EncodingABI`, delegates to `EncodeABIGMPPacketData`, which performs Solidity ABI encoding
- `k.msgRouter.Handler(msg)` — router lookup, which resolves the registered `MsgServiceHandler` for `*MsgSendPacket`
- `handler(ctx, msg)` — invokes the IBC core V2 `SendPacket` message handler, which
  - revalidates the `MsgSendPacket` fields (client identifier, time-out, payload);
  - verifies the source client is active and not expired;
  - writes a packet commitment hash to the IBC core KV store (keyed by `(sourceClient, sequence)`);
  - increments the per-client send sequence counter;
  - creates a new internal `EventManager` for its scope, emitting `EventTypeSendPacket` events; and
  - returns a `sdk.Result` containing exactly one `MsgSendPacketResponse` with the assigned Sequence
- `ctx.EventManager().EmitEvents(res.GetEvents())` — reemits events from the handler's internal `EventManager` into the outer context
- `proto.Unmarshal(msgResponse.Value, &sendResponse)` — deserializes the raw packed response bytes into `MsgSendPacketResponse`

## IBC Module

**Function:** `OnSendPacket(ctx sdk.Context, sourceChannel string, destinationChannel string, sequence uint64, payload channeltypesv2.Payload, signer sdk.AccAddress)`

*Module:* `modules/apps/27-gmp/ibc_module.go`

### Intended behavior

The `OnSendPacket` function is the callback invoked by the IBC core routing module immediately before a GMP packet is committed to state on the source chain. Its role is to perform a final layer of application-level validation on the outgoing payload verifying port bindings, client-identifier format, packet data integrity, and signer authority and to emit a send-call event.

### Preconditions

- Both `payload.SourcePort` and `payload.DestinationPort` must equal `"gmpport"`.
- Both `sourceChannel` and `destinationChannel` must be valid IBC client identifiers in the format `{string}-{number}`.
- The `payload.Value` must be a valid serialized `GMPPacketData` that passes `ValidateBasic`.
- The sender address embedded in the packet data must be a valid Bech32 address and must equal the signer argument.

### Inputs

- `sourceChannel string` — the local IBC client identifier on the source chain
- `destinationChannel string` — the counterparty IBC client identifier
- `sequence uint64` — the packet sequence number assigned by IBC core
- `payload channeltypesv2.Payload` — the IBC V2 payload containing the GMP packet data, with relevant subfields:
  - `SourcePort string` — must be `"gmpport"`
  - `DestinationPort string` — must be `"gmpport"`
  - `Version string` — must be `"ics27-2"`
  - `Encoding string` — the serialization format used for `payload.Value`
  - `Value []byte` — the serialized `GMPPacketData`
- `signer sdk.AccAddress` — the address that signed the originating `MsgSendPacket` — must match the sender encoded in the packet data

### Branching behavior

- The port IDs on both the source and destination sides of the payload are validated first.
- Both `sourceChannel` and `destinationChannel` are validated as properly formatted IBC client identifiers.
- The payload bytes are deserialized into `GMPPacketData` using the encoding specified in the payload.
- The deserialized packet data is validated against field constraints (size limits, nonempty sender, etc.).
- The sender address from the packet data is decoded as a Bech32 address and compared against the `signer` argument.
- If all checks pass, a `SendCall` event is emitted and `nil` is returned.

### External call analysis

- `types.UnmarshalPacketData(...)` — stateless deserialization of the payload bytes into `GMPPacketData` — dispatches to JSON, protobuf, or ABI decoding based on `payload.Encoding`
- `data.ValidateBasic()` — stateless field-level validation of the deserialized packet data
- `events.EmitSendCall(...)` — emits a `EventTypeSendCall` event and a generic message-module event into the context's `EventManager`

**Function: `OnRecvPacket(ctx sdk.Context, sourceClient, destinationClient string, sequence uint64, payload channeltypesv2.Payload, relayer sdk.AccAddress)`**

*Module:* `modules/apps/27-gmp/ibc_module.go`

### Intended behavior

The `OnRecvPacket` function is the callback invoked on the destination chain when a GMP packet is delivered by a relayer. It validates the incoming payload, derives or creates the deterministic GMP account for the `(destClient, sender, salt)` triplet, executes the encoded Cosmos SDK messages as that account, serializes the execution result into an `Acknowledgement`, and returns it to the IBC core for commitment. All application-level errors produce a `PacketStatus_Failure` result rather than reverting the IBC state, as per IBC V2 semantics. A deferred event is always emitted regardless of the execution outcome.

### Preconditions

- Both `payload.SourcePort` and `payload.DestinationPort` must equal `"gmpport"`.

- The `payload.Version` must equal `"ics27-2"`.
- The `payload.Value` must deserialize into a valid `GMPPacketData`.
- The payload bytes must decode successfully under `payload.Encoding`.
- The GMP account creation and address derivation must succeed.
- The deserialized payload must contain at least one SDK message, and all messages must declare exactly one signer, which must match the derived GMP account address.

### Inputs

- `sourceClient` string — the IBC client identifier on the destination chain tracking the source chain, used as part of the `(destClient, sender, salt)` triplet for deterministic account derivation
- `destinationClient` string — the local IBC client identifier, carried through to events and logging
- `sequence` uint64 — packet sequence number, used in logging and events
- `payload` `channeltypesv2.Payload` — the IBC V2 payload, with relevant subfields:
  - `SourcePort` string — must be `"gmpport"`
  - `DestinationPort` string — must be `"gmpport"`
  - `Version` string — must be `"ics27-2"`
  - `Encoding` string — the serialization format for both packet data and the returned acknowledgment
  - `Value []byte` — the serialized `GMPPacketData`
- `relayer` `sdk.AccAddress`: The relayer address that submitted the packet.

### Branching behavior

- A deferred closure is registered at the start to unconditionally emit a `RecvPacket` event when the function returns. The event reflects the final `ackErr` value, indicating success or failure.
- The source and destination port IDs are validated; a mismatch sets `ackErr` and returns `PacketStatus_Failure`.
- The payload version is validated against `"ics27-2"`; a mismatch sets `ackErr` and returns `PacketStatus_Failure`.
- The payload bytes are deserialized; a failure sets `ackErr` and returns `PacketStatus_Failure`. On success, data is assigned so the deferred event emitter captures the decoded packet fields.
- The deserialized packet data is validated; a failure sets `ackErr` and returns `PacketStatus_Failure`.
- The keeper's `OnRecvPacket` is called, which derives or creates the GMP account and executes the payload messages (see below external call analysis); any error sets `ackErr` and returns `PacketStatus_Failure`.

- The execution result bytes are wrapped into an Acknowledgement and serialized using the same encoding as the incoming packet; a serialization failure sets `ackErr` and returns `PacketStatus_Failure`.
- On full success, an info log is emitted and a `PacketStatus_Success` result carrying the serialized acknowledgment bytes is returned.

### External call analysis

- `events.EmitOnRecvPacketEvent(...)` (deferred) — always fires on function exit and emits an `EventTypeRecvPacket` event with packet attributes plus an `ack_success` boolean and, on failure, an `ack_error` string — no state mutation.
- `types.UnmarshalPacketData(...)` — stateless deserialization, no state reads or writes
- `packetData.ValidateBasic()` — stateless field validation, no state reads or writes
- `im.keeper.OnRecvPacket(ctx, packetData, destinationClient)` — this does the following:
  - It constructs an `AccountIdentifier` from `(destClient, sender, salt)`.
  - It calls `getOrCreateICS27Account`, which first checks the keeper's `Accounts` collection for an existing entry. If not found, it derives the address deterministically using `BuildAddressPredictable`. If no base account exists at that address, it creates a new base account via `accountKeeper.NewAccountWithAddress` and persists it. Either way, the `Accounts` and `AccountsByAddress` collection entries are written if a new GMP account was registered.
  - It retrieves the materialized account from the `accountKeeper`.
  - It calls `executeTx`, which deserializes the payload into a list of Cosmos SDK `Msgs`, verifies each message is signed by the GMP account address, then executes them atomically in a cache context — state is only committed if every message succeeds.
  - It returns the protobuf-serialized `TxMsgData` containing each message's response.
- `types.MarshalAcknowledgement(...)` — stateless serialization of the `Acknowledgement` into bytes using the same encoding as the inbound packet — panics if the version is not "ics27-2" (invariant)

**Function: `OnTimeoutPacket(_ sdk.Context, _, _ string, _ uint64, _ channeltypesv2.Payload, _ sdk.AccAddress) error`**

*Module:* `modules/apps/27-gmp/ibc_module.go`

### Intended behavior

The `OnTimeoutPacket` function is the callback invoked on the source chain when a sent GMP packet expires without being received. In the current implementation (as of the time of writing), it is a no-op.

### Preconditions

None. The function accepts all inputs and returns unconditionally.

### Inputs

All parameters are explicitly ignored.

- Two `string` parameters — source and destination client identifiers
- `uint64` — packet sequence number
- `channeltypesv2.Payload` — the timed-out packet payload
- `sdk.AccAddress` — relayer address

### Branching behavior

- Unconditionally returns `nil`. No branching.

### External call analysis

None.

**Function:** `OnAcknowledgementPacket(_ sdk.Context, _, _ string, _ uint64, _ []byte, _ channeltypesv2.Payload, _ sdk.AccAddress) error`

*Module:* `modules/apps/27-gmp/ibc_module.go`

### Intended behavior

The `OnAcknowledgementPacket` function is the ICS-26 callback invoked on the source chain after the destination chain's acknowledgment is committed. In the current implementation (as of the time of writing), it is a no-op.

### Preconditions

None. The function accepts all inputs and returns unconditionally.

### Inputs

All parameters are explicitly ignored.

- `sdk.Context` — execution context
- Two `string` parameters — source and destination client identifiers
- `uint64` — packet sequence number
- `[]byte` — the raw acknowledgment bytes from the destination
- `channeltypesv2.Payload` — the original packet payload
- `sdk.AccAddress` — relayer address

### Branching behavior

- Unconditionally returns `nil`. No branching.

### External call analysis

None.

### Function: `UnmarshalPacketData(payload channeltypesv2.Payload)`

*Module:* `modules/apps/27-gmp/ibc_module.go`

### Intended behavior

The `UnmarshalPacketData` function implements the `PacketDataUnmarshaler` interface required by the ICS-30 callbacks middleware. It provides a standardized entry point through which middleware can decode a raw IBC V2 payload into a typed `GMPPacketData` value — for example, to extract the sender address for automatic source callback registration without the middleware needing to understand GMP-specific encoding details.

### Preconditions

- The `payload.Value` must be a valid serialized `GMPPacketData` compatible with `payload.Encoding`.
- The `payload.Version` must equal `"ics27-2"`.

### Inputs

- `payload channeltypesv2.Payload` — the IBC V2 payload to decode, with relevant subfields:
  - `Value []byte` — the serialized `GMPPacketData` bytes
  - `Version string` — must be "ics27-2"
  - `Encoding string` — the serialization format ("application/json", "application/x-protobuf", or "application/x-solidity-abi")

### Branching behavior

- The payload bytes are deserialized according to the encoding; any failure returns `(nil, err)`.
- On success, the typed `*GMPPacketData` is returned as an any interface value.

### External call analysis

- `types.UnmarshalPacketData(payload.Value, payload.Version, payload.Encoding)` — stateless deserialization, identical to the call in `OnSendPacket` and `OnRecvPacket`, and dispatches to JSON, protobuf, or ABI decoding

## 4.2. Module: Attestations light client

This section provides a comprehensive threat model for the attestations light client, detailing the intended behavior, preconditions, inputs, and potential risks for the light-client module's core functions.

### Attestations light client overview

The attestations light client is a custom IBC light-client implementation that replaces conventional blockchain header verification with a multi-sig attestation scheme. Rather than verifying cryptographic proofs derived from a counterparty chain's block headers, it relies on a quorum of off-chain attestors — identified by their `Secp256k1` public keys — to sign state and packet data attestations. A client instance is parameterized by an attesor set (`AttestorAddresses`), a quorum threshold (`MinRequiredSigs`), and an initial height and timestamp. The client state advances when a threshold-satisfying `StateAttestation` is submitted, and membership or nonmembership proofs are verified by checking that a `PacketAttestation` covering the queried key-value pair carries the same quorum-level signature.

## LightClientModule

**Function:** `Initialize(ctx sdk.Context, clientStore storetypes.KVStore, cdc codec.BinaryCodec, clientState exported.ClientState, consensusState exported.ConsensusState) error`

*Module:* `modules/light-clients/attestations/light_client_module.go`

### Intended behavior

The `Initialize` function is called by the IBC core once when a new light-client instance is created. It casts the generic `ClientState` and `ConsensusState` interfaces to the attestation-specific types, validates the client state fields, and persists both the client state and the initial consensus state to the client store.

### Preconditions

- `clientState` must be castable to `*ClientState`.
- `consensusState` must be castable to `*ConsensusState`.
- `clientState.LatestHeight` must be nonzero.
- `clientState.MinRequiredSigs` must be greater than zero.
- `len(clientState.AttestorAddresses)` must be at least `clientState.MinRequiredSigs`.
- Each entry in `AttestorAddresses` must be a valid checksummed EVM address.

### Inputs

- `clientState exported.ClientState` — the initial client state, with relevant fields:
  - `LatestHeight uint64` — the initial attested height, must be nonzero
  - `MinRequiredSigs uint32` — the quorum threshold for attester signatures
  - `AttestorAddresses []string` — EVM-style hex addresses of the authorized attestors
  - `IsFrozen bool` — must be false on initialization
- `consensusState exported.ConsensusState` — the initial consensus state, with a relevant field:
  - `Timestamp uint64` — the timestamp corresponding to `LatestHeight`

### Branching behavior

- The `clientState.Validate()` is called; any validation failure returns an error

immediately.

- The initial consensus state is stored at `clientState.LatestHeight`.
- The client state is stored.

### External call analysis

- `clientState.Validate()` — stateless field-level validation, checks `LatestHeight > 0`, `MinRequiredSigs > 0`, and that the attester set has at least `MinRequiredSigs` entries
- `setConsensusState(clientStore, cdc, consensusState, height)` — encodes and writes the consensus state to the client store under the height key
- `setClientState(clientStore, cdc, cs)` — encodes and writes the client state to the client store

**Function:** `VerifyClientMessage(ctx sdk.Context, clientStore storetypes.KVStore, cdc codec.BinaryCodec, clientMsg exported.ClientMessage) error`

*Module:* `modules/light-clients/attestations/light_client_module.go`

### Intended behavior

The `VerifyClientMessage` function validates an incoming `AttestationProof` submitted as a `ClientMessage` before `UpdateState` is allowed to apply it. It checks that the client is not frozen, reads the current client state, and delegates signature verification to `verifySignatures`. It does not apply any state changes.

### Preconditions

- The `clientMsg` must be castable to `*AttestationProof`.
- The client must not be frozen (`cs.IsFrozen == false`).
- The `proof.AttestationData` must be a nonempty byte slice.
- At least `cs.MinRequiredSigs` valid ECDSA signatures over `sha256(proof.AttestationData)` must be present in `proof.Signatures`, each recovering to a distinct address in `cs.AttesterAddresses`.

### Inputs

- `clientMsg exported.ClientMessage` — the proof to verify, cast to `*AttestationProof`:
  - `AttestationData []byte` — the raw ABI-encoded attestation blob (either `StateAttestation` or `PacketAttestation`) — the function does not inspect

the content but verifies the signature over the raw bytes

- Signatures `[][]byte` – raw ECDSA signatures (65-byte `r || s || v` format).

### Branching behavior

- The client state is read from the store; a missing state is an error.
- If `cs.IsFrozen` is true, an error is returned immediately.
- Additionally, `verifySignatures(cs, proof)` is called; any failure returns an error.

### External call analysis

- `getClientState(clientStore, cdc)` – reads and decodes the client state from the KV store – read-only
- `verifySignatures(cs, proof)` – performs ECDSA public-key recovery and quorum threshold check – see dedicated section below

**Function: `UpdateState(ctx sdk.Context, clientStore storetypes.KVStore, cdc codec.BinaryCodec, clientMsg exported.ClientMessage) []exported.Height`**

*Modules:* `modules/light-clients/attestations/light_client_module.go` and `modules/light-clients/attestations/update.go`

### Intended behavior

The `UpdateState` function is called by IBC core after `VerifyClientMessage` succeeds. It ABI-decodes the `AttestationData` as a `StateAttestation`, creates a new `ConsensusState` from the decoded timestamp, stores it at the attested height, and conditionally advances `LatestHeight` if the attested height exceeds the current value.

### Preconditions

- `VerifyClientMessage` has been called and returned `nil` for the same `clientMsg` in the same block.
- `proof.AttestationData` must be ABI-decodable as a `StateAttestation` (`(uint64 Height, uint64 Timestamp)`); a decode failure panics.

### Inputs

- `clientMsg exported.ClientMessage` — cast to `*AttestationProof` — only `AttestationData` is used here:
  - `AttestationData []byte` — must ABI-decode as `StateAttestation`

### Branching behavior

- The client state is read; a missing state is an error (propagates as panic in the IBC core path).
- `ABIDecodeStateAttestation(proof.AttestationData)` is called; a failure panics unconditionally rather than returning an error.
- A `ConsensusState` is constructed with `Timestamp = stateAttestation.Timestamp`.
- The consensus state is stored at `clienttypes.NewHeight(0, stateAttestation.Height)`.
- If `stateAttestation.Height > cs.LatestHeight`, `cs.LatestHeight` is updated.
- No zero-height guard is present; a quorum-signed `stateAttestation.Height == 0` would store a consensus state at height zero and potentially set `LatestHeight` to zero (if the current `LatestHeight` is also zero), in inconsistency with `Initialize` and `Validate`.
- The updated client state is stored, and the new height is returned.

### External call analysis

- `getClientState(clientStore, cdc)` — reads the current client state — read-only
- `ABIDecodeStateAttestation(attestationData)` — stateless ABI decoding, panics on malformed input rather than returning an error
- `setConsensusState(clientStore, cdc, consensusState, height)` — writes the new consensus state to the store
- `setClientState(clientStore, cdc, cs)` — writes the updated client state to the store

**Function:** `CheckForMisbehaviour(ctx sdk.Context, clientStore storetypes.KVStore, cdc codec.BinaryCodec, clientMsg exported.ClientMessage) bool`

*Module:* `modules/light-clients/attestations/light_client_module.go`

### Intended behavior

The `CheckForMisbehaviour` function is the IBC misbehavior-detection hook. In the implementation as of the time of writing, it is a stub that unconditionally returns `false`. No misbehavior-detection

logic is implemented; the client cannot detect conflicting attestations submitted by a compromised attester set.

#### Preconditions

None. The function accepts all inputs.

#### Inputs

N/A

#### Branching behavior

- Unconditionally returns `false`. No branching.

#### External call analysis

None.

**Function:** `UpdateStateOnMisbehaviour(ctx sdk.Context, clientStore storetypes.KVStore, cdc codec.BinaryCodec, clientMsg exported.ClientMessage)`

*Module:* `modules/light-clients/attestations/light_client_module.go`

#### Intended behavior

The `UpdateStateOnMisbehaviour` function is the IBC hook invoked after `CheckForMisbehaviour` returns `true`. Because `CheckForMisbehaviour` always returns `false`, this function is unreachable in practice. It panics unconditionally if called.

#### Preconditions

Unreachable under the logic as of the time of writing.

#### Inputs

N/A

### Branching behavior

- Unconditionally panics with `ErrInvalidRequest`.

### External call analysis

None.

**Function:** `VerifyMembership(ctx sdk.Context, clientStore store-types.KVStore, cdc codec.BinaryCodec, height exported.Height, delay-  
TimePeriod, delayBlockPeriod uint64, proof []byte, path exported.Path,  
value []byte) error`

*Module:* `modules/light-clients/attestations/light_client_module.go`

### Intended behavior

The `VerifyMembership` function verifies that a given key-value pair was attested by a quorum of attestors at the specified height. It checks the frozen status, retrieves the consensus state stored at the target height, decodes the proof as an `AttestationProof`, verifies the quorum signatures over the raw `AttestationData`, ABI-decodes the data as a `PacketAttestation`, and checks whether the queried (`path`, `value`) pair appears in the attested packet list.

### Preconditions

- The client must not be frozen.
- A consensus state must exist at the requested height.
- The proof must deserialize into a valid `AttestationProof`.
- The `proof.AttestationData` must ABI-decode as a `PacketAttestation`.
- A quorum of valid ECDSA signatures over `sha256(proof.AttestationData)` must be present.
- The (`path`, `value`) pair must appear in the `PacketAttestation.Packets` list.

### Inputs

- `height exported.Height` — the height at which the membership proof is asserted — a consensus state must exist at this height
- `proof []byte` — proto-encoded `AttestationProof`
- `path exported.Path` — the IBC commitment path being proven
- `value []byte` — the expected value at `path`

### Branching behavior

- The client state is read; a frozen client returns an error immediately.
- The consensus state at height is read; a missing entry returns an error.
- The proof bytes are protodecoded into `AttestationProof`; a decode failure returns an error.
- `verifySignatures(cs, proof)` is called; a quorum failure returns an error.
- `ABIDecodePacketAttestation(proof.AttestationData)` is called; a decode failure returns an error.
- The decoded packet list is searched for an entry matching `(path, value)`; no match returns an error.

### External call analysis

- `getClientState(clientStore, cdc)` — read-only client state retrieval
- `getConsensusState(clientStore, cdc, height)` — read-only consensus state retrieval at the target height
- `verifySignatures(cs, proof)` — ECDSA recovery and quorum check — see dedicated section below
- `ABIDecodePacketAttestation(attestationData)` — stateless ABI decoding of the `PacketAttestation` struct

**Function:** `VerifyNonMembership(ctx sdk.Context, clientStore storetypes.KVStore, cdc codec.BinaryCodec, height exported.Height, delayTimePeriod, delayBlockPeriod uint64, proof []byte, path exported.Path) error`

*Module:* `modules/light-clients/attestations/light_client_module.go`

### Intended behavior

The `VerifyNonMembership` function mirrors `VerifyMembership` but asserts the absence of a key at the given height. It follows the same validation path — frozen check, consensus state lookup, signature verification, ABI decoding — and then confirms that no entry for `path` appears in the `PacketAttestation.Packets` list.

### Preconditions

- The client must not be frozen.
- A consensus state must exist at the requested height.
- The `proof.AttestationData` must ABI-decode as a `PacketAttestation`.

- A quorum of valid ECDSA signatures must be present.
- The path must not appear in the `PacketAttestation.Packets` list.

### Inputs

- `height exported.Height` — the height at which nonmembership is asserted
- `proof []byte` — proto-encoded `AttestationProof`
- `path exported.Path` — the IBC commitment path asserted to be absent

### Branching behavior

- Identical to `VerifyMembership` up to the final lookup step.
- The decoded packet list is searched for an entry matching path; a match returns an error (presence when absence is expected).

### External call analysis

Same as `VerifyMembership` above.

**Function:** `Status(ctx sdk.Context, clientStore storetypes.KVStore, cdc codec.BinaryCodec) exported.Status`

*Module:* `modules/light-clients/attestations/light_client_module.go`

### Intended behavior

The `Status` function returns the current operational status of the light client. It reads the client state and returns `Frozen` if `cs.IsFrozen` is `true`, or `Active` otherwise. Because no code path sets `IsFrozen` to `true` at runtime (see Finding 3.3.7), this function always returns `Active` in practice.

### Preconditions

None beyond a readable store.

### Inputs

N/A

### Branching behavior

- If the client state cannot be read, Unknown is returned.
- If `cs.IsFrozen`, it returns Frozen. Otherwise, it returns Active.

### External call analysis

- `getClientState(clientStore, cdc)` — read-only

**Function: LatestHeight(`ctx sdk.Context`, `clientStore storetypes.KVStore`, `cdc codec.BinaryCodec`) `exported.Height`**

*Module:* `modules/light-clients/attestations/light_client_module.go`

### Intended behavior

The LatestHeight function returns the highest attested height stored in the client state as a `clienttypes.Height` with revision number zero.

### Preconditions

None beyond a readable store.

### Inputs

N/A

### Branching behavior

- If the client state cannot be read, a zero height is returned. Otherwise, it returns `clienttypes.NewHeight(0, cs.LatestHeight)`.

### External call analysis

- `getClientState(clientStore, cdc)` — read-only

## Signature verification

**Function:** `verifySignatures(cs *ClientState, proof *AttestationProof) error`

*Module:* `modules/light-clients/attestations/signature.go`

### Intended behavior

The `verifySignatures` function is the shared cryptographic verification primitive used by `VerifyClientMessage`, `VerifyMembership`, and `VerifyNonMembership`. It computes `hash = sha256(proof.AttestationData)`, iterates over `proof.Signatures` to recover each ECDSA public key, checks each recovered address against `cs.AttestorAddresses`, enforces uniqueness within a single call, and finally asserts that the number of valid signatures meets `cs.MinRequiredSigs`. The hash is computed directly over raw `AttestationData` without a domain separator or client-identifier binding.

### Preconditions

- The `proof.AttestationData` must be a nonempty byte slice.
- Each entry in `proof.Signatures` must be a 65-byte raw ECDSA signature in `r || s || v` format.
- At least `cs.MinRequiredSigs` of the recovered addresses must appear in `cs.AttestorAddresses`.
- No recovered address may appear more than once within `proof.Signatures`.

### Inputs

- `cs *ClientState` — provides `AttestorAddresses` and `MinRequiredSigs`
- `proof *AttestationProof` — provides the blob to verify (`AttestationData`) and the signatures (`Signatures`)

### Branching behavior

- `hash = sha256(proof.AttestationData)` is computed once before the loop. The hash carries no domain separator identifying the attestation type (`StateAttestation` vs `PacketAttestation`) or the client instance.
- For each signature, `crypto.SigToPub(hash[:], sig)` is called to recover the public key; a recovery failure skips that signature (or returns an error, depending on implementation).
- The recovered address is checked for presence in `cs.AttestorAddresses` and for absence in the current-call seen set; a duplicate or non-attestor address does not

increment the valid-signature count.

- After the loop, `len(proof.Signatures) < int(cs.MinRequiredSigs)` is checked; if the quorum is not met, an error is returned. This check occurs after all ECDSA recoveries, meaning under-quorum proofs trigger full recovery work before being rejected.

### External call analysis

- `sha256.Sum256(proof.AttestationData)` — stateless hash computation, with no domain separator — the same hash function for both `StateAttestation` and `PacketAttestation` blobs
- `crypto.SigToPub(hash[:], sig)` — ECDSA public-key recovery and is stateless — cost is linear in the number of signatures provided

## ClientState validation

### Function: (`cs *ClientState`) `Validate()` error

*Module:* `modules/light-clients/attestations/client_state.go`

### Intended behavior

The `Validate` function performs stateless field-level validation of a `ClientState` value. It is called during `Initialize` and may be called independently during client state import or upgrade.

### Preconditions

None. The function validates the struct in isolation.

### Inputs

- `cs.LatestHeight uint64` — must be greater than zero
- `cs.MinRequiredSigs uint32` — must be greater than zero
- `cs.AttestorAddresses []string` — must have at least `MinRequiredSigs` entries, and each must be a valid EVM hex address
- `cs.IsFrozen bool` — no constraint enforced by `Validate`

### Branching behavior

- Returns an error if `LatestHeight == 0`.
- Returns an error if `MinRequiredSigs == 0`.

- Returns an error if `len(AttestorAddresses) < MinRequiredSigs`.
- Returns an error if any `AttestorAddresses` entry fails EVM address format validation.
- Returns `nil` if all checks pass.

### External call analysis

No external calls. Fully stateless.

### ABI encoding

**Function:** `ABIDecodeStateAttestation(data []byte) (StateAttestation, error)`

*Module:* `modules/light-clients/attestations/abi.go`

### Intended behavior

The `ABIDecodeStateAttestation` function decodes a Solidity ABI-encoded `StateAttestation` struct (`uint64 Height`, `uint64 Timestamp`) from raw bytes. It is called unconditionally by `UpdateState`; a decode failure panics rather than propagating an error.

### Preconditions

N/A

### Inputs

- `data []byte` — raw ABI-encoded bytes — expected layout is a (`uint64`, `uint64`) tuple

### Branching behavior

- Returns a decoded `StateAttestation` on success.
- Returns an error on malformed input — callers in `UpdateState` do not handle this error and panic.

### External call analysis

N/A

**Function: `ABIDecodePacketAttestation(data []byte) (PacketAttestation, error)`**

*Module:* modules/light-clients/attestations/abi.go

**Intended behavior**

The `ABIDecodePacketAttestation` function decodes a Solidity ABI-encoded `PacketAttestation` struct (`uint64 Height, PacketCompact[] Packets`) from raw bytes. It is called by `VerifyMembership` and `VerifyNonMembership` after signature verification.

**Preconditions**

N/A

**Inputs**

- `data []byte` — raw ABI-encoded bytes — expected layout is (`uint64, (bytes32, bytes32, uint64)[]`)

**Branching behavior**

- Returns a decoded `PacketAttestation` on success.
- Returns an error on malformed input — callers propagate the error.

**External call analysis**

N/A

**Notes on cross-type confusion**

Both `ABIDecodeStateAttestation` and `ABIDecodePacketAttestation` operate on blobs signed with the same untagged sha256 hash (see `verifySignatures`). A `PacketAttestation` blob is structurally decodable as a `StateAttestation`; the ABI dynamic-type offset pointer occupying the first 32 bytes decodes to a fixed value of 32 as `StateAttestation.Height`, and the attested height within the `PacketAttestation` body decodes as `StateAttestation.Timestamp`. This structural overlap is the root cause of Finding [3.1](#).

## 5. Assessment Results

During our assessment on the scoped IBC-Go GMP Module and Attestations Light Client modules, we discovered nine findings. No critical issues were found. One finding was of high impact, two were of medium impact, two were of low impact, and the remaining findings were informational in nature.

---

### 5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.